ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 弧月建設株式会社 3 年半の航海に投入された Python 実装 ―

本書は、ケーススタディ本編「弧月の測地線」に登場する Python コード断片を、
パイプラインの 10 ステップ構造に沿って整理し、
各コード片の数学的背景と運用上の意図を解説した実装ガイドである。

Étale Cohomology

2026 年 10 月

License: CC BY 4.0

本書の位置づけ

本書はケーススタディ「弧月の測地線」の補助文書 03 である。関連文書として以下がある。

- 本編：「弧月の測地線 ― 弧月建設株式会社 3 年半の航海」
- 補助文書 01：本編と GI 理論の対応関係
- 補助文書 02：本編で使用された数学の解説
- 本書（補助文書 03）：実装スクリプト集
- 補助文書 04：業界知識

掲載コードについて

本書に掲載したコード断片は、本編に登場した擬似コードを忠実に再録したものである。本編の擬似コードは、物語の中で数式とコードを橋渡しすることを優先しているため、そのままでは実行できない箇所（外部関数の呼び出し、省略された数学的処理、本編の数値を出力として示した部分など）を含む。

また、教育目的の簡略化のために、数学的に厳密でない箇所もある。本書ではそれらを書き換えず、各コード片の「実装上の注意」で指摘し、正しい実装の考え方を補足した。主な補足の参照実装は付録 B にまとめている。本編の物語の数値は、本書の注意を反映した実装による再計算の結果ではない点に留意されたい。

ライセンスとクレジット

© 2026 Étale Cohomology. 本書は Creative Commons Attribution 4.0 International（CC BY 4.0）のもとで公開される。

目次

1　はじめに　4

1.1　パイプライン全体像 4
1.2　本作のコードの特徴 4
1.3　各コード片の構成 4
1.4　推奨ライブラリ 4

2　ステップ 1〜2：データ収集と多様体構築　5

2.1　固有次元数の推定 5
2.2　命題 1.1 の検証 — 第 1 回試行（不合格） 6
2.3　プロジェクト共分散を保存した月次企業状態ベクトル 7
2.4　命題 1.1 の検証 — 第 2 回試行 8
2.5　技術者異動データの読み込み 9
2.6　制約付き Neural ODE による地理的レジーム間の遷移補間 10
2.7　命題 1.1 の検証 — 第 3 回試行（合格） 11

3　ステップ 3：引き戻し計量の計算　12

3.1　引き戻し計量と条件数 12

4　ステップ 4：スカラー曲率と測地線距離　14

4.1　スカラー曲率の計算 14
4.2　測地線距離とユークリッド距離の比較 15

5　AI エージェントの設計と身体化エージェント　17

5.1　14 体の AI エージェント・ロスター 17
5.2　身体化エージェントのアラート（タワークレーン 8 号機） 19
5.3　非身体化エージェントのアラート（サイバー・ガーディアン） 20
5.4　身体化 × 非身体化のリー括弧 21
5.5　物理的ベースライン・チェック 22

6　ステップ 5：リー微分とリー括弧　24

6.1　NEOM 無期限中断のリー微分 24
6.2　リー括弧の計算関数（NEOM 中断 × サイバー探索） 25
6.3　3 大国の動きと三重リー括弧 27
6.4　6 つの危機の全ペアのリー括弧行列 28

7　ステップ 8：最適制御　31

7.1　NEOM 危機の緊急最適制御 31
7.2　インドネシア贈賄危機の最適制御 32

8　イベント間の幾何学とエージェントのアラート　34
8.1　NEOM 中断とインドネシア規制変更の測地線距離 34
8.2　5 つのイベントの測地線距離行列 34
8.3　エージェントのアラート形式 35

9　ステップ 7：DML による因果分解　37
9.1　鉄鋼価格上昇の因果分解 37

10 幾何学的拡張　39
10.1 EU CBAM のベクトル場と 7 つ目の点 39
10.2 パーシステント・ホモロジー 39
10.3 多人数ゲーム理論 41
10.4 手術理論によるポートフォリオの位相変換 43
10.5 シンプレクティック積分による 7 年間のシミュレーション 45
10.6 同盟構築後の多様体の更新 47
10.7 層理論による内部情報漏洩の検知 47
10.8 座標系の変換（本編の呼称：ゲージ変換） 48

11 形式検証と ZKML　50
11.1 ZKML 証明の生成 50

付録 A　推奨ライブラリ一覧　51

付録 B　補足の参照実装　52

付録 C　コード索引　54

第1章　はじめに

1.1　パイプライン全体像

GI 理論のパイプラインは、データ収集（ステップ 1）、VAE による多様体構築（2）、引き戻し計量（3）、曲率（4）、リー微分・リー括弧（5）、MC ドロップアウト（6）、SHAP と DML（7）、Neural ODE と最適制御（8）、品質チェック（9）、レポート生成（10）の 10 ステップから成る。本書の章立てはこの順序に従い、その後に AI エージェント、幾何学的拡張、形式検証と ZKML を置いた。

1.2　本作のコードの特徴

1 作目「生存の幾何学」のコードと比べ、本作には次の特徴がある。

- **プロジェクト型データの変換**。プロジェクト単位のデータを月次の企業状態ベクトルに変換する関数（第 2.3 節）と、地理的レジーム間の遷移を制約付き Neural ODE で補間するクラス（第 2.6 節）。
- **身体化エージェント**。建設現場のセンサーのアラート形式と、物理法則に基づくベースライン・チェック（第 5 章）。
- **イベント間の幾何学**。危機どうしの測地線距離行列、リー括弧行列、パーシステント・ホモロジー（第 7〜8 章、第 10 章）。

1.3　各コード片の構成

各節は「背景」「コード」「解説」「実装上の注意」の順に構成する。「背景」は本編のどの場面でそのコードが登場したか、「解説」はコードが何を計算しているか、「実装上の注意」は実運用で修正・補強すべき点を述べる。

1.4　推奨ライブラリ

本編のコードは、NumPy、PyTorch、torchdiffeq、scikit-learn、EconML、ripser / persim を前提としている。詳細は付録 A を参照。

第2章　ステップ 1〜2：データ収集と多様体構築

2.1　固有次元数の推定

出典：本編第 2 章

背景　本編第 2 章「命題 1.1 の壁」。VAE の潜在次元を決める前に、120 指標のデータの「実効的な自由度」を推定する場面。

コード

```
import torch
import torch.nn as nn
import numpy as np
from sklearn.neighbors import NearestNeighbors

def estimate_intrinsic_dimension(X, k_range=(5, 50)):
    """
    固有次元数の最尤推定。手法：Levina-Bickel (2004)

    ◆ 何をしているか：
      120次元のデータの中で、本当に独立に動いている軸が
      何本あるかを推定する。多くの指標は相関しているため
      （例：鉄鋼価格↑ → 建設コスト↑ → 利益率↓）、
      実効的な次元数は生の指標数よりずっと少ない。
    """
    n_samples = X.shape[0]
    dims = []

    for k in range(k_range[0], k_range[1]):
        nn_model = NearestNeighbors(n_neighbors=k+1).fit(X)
        distances, _ = nn_model.kneighbors(X)

        T_k = distances[:, k]
        local_dims = []
        for i in range(n_samples):
            if T_k[i] > 0:
                log_ratios = np.log(T_k[i] / distances[i, 1:k])
                d_hat = (k - 1) / np.sum(log_ratios)
                local_dims.append(d_hat)
        dims.append(np.median(local_dims))

    return np.median(dims)

d_hat = estimate_intrinsic_dimension(X_normalized_kogetsu)
print(f"推定固有次元数：{d_hat:.1f}")
推定固有次元数：11.7
```

解説　Levina–Bickel（2004）の最尤推定量を、近傍数 k を 5〜49 の範囲で変えながら計算し、その中央値をとっている。各点で k 番目の近傍までの距離 T_k と、それより近い近傍までの距離の対数比から局所次元を推定する。弧月のデータでは 11.7 という値が得られ、潜在次元を 12 とする根拠になった。

実装上の注意

k の範囲によって推定値は変わりうる。本編のように k を走査して中央値をとるのは良い実践である。データに重複点がある場合は距離が 0 になり対数比が発散するため、事前に重複を除くか、距離 0 の近傍を除外する処理が必要である。また、TwoNN（Facco et al., 2017）など別の推定量でも確認し、値が大きく食い違わないかを見るとよい。

2.2　命題 1.1 の検証 ― 第 1 回試行（不合格）

出典：本編第 2 章

背景　同じく第 2 章。プロジェクトごとのデータをそのまま VAE に入力した最初の試行の結果。

コード

```
# ===== 命題1.1検証結果 ―― 弧月建設 第1回試行 =====

# 条件1（再構成忠実度）：
mse_reconstruction = 0.0068
threshold_epsilon1 = 0.01
condition_1 = mse_reconstruction < threshold_epsilon1
print(f"条件1：MSE = {mse_reconstruction:.4f} < {threshold_epsilon1}")
print(f"  → {'合格 ✓' if condition_1 else '不合格 ×'}")

# 条件2（潜在空間の滑らかさ）：
knn_preservation_rate = 0.72
threshold_epsilon2 = 0.90
condition_2 = knn_preservation_rate >= threshold_epsilon2
print(f"条件2：KNN保存率 = {knn_preservation_rate:.2f} >= {threshold_epsilon2}")
print(f"  → {'合格 ✓' if condition_2 else '不合格 ×'}")

# 条件3（ヤコビアンの非退化性）：
min_singular_value = 0.00007
threshold_epsilon3 = 0.001
condition_3 = min_singular_value >= threshold_epsilon3
print(f"条件3：最小特異値 = {min_singular_value:.5f} >= {threshold_epsilon3}")
print(f"  → {'合格 ✓' if condition_3 else '不合格 ×'}")
条件1：MSE = 0.0068 < 0.01
  → 合格 ✓
条件2：KNN保存率 = 0.72 >= 0.90
  → 不合格 ×
```

```
条件3：最小特異値 = 0.00007 >= 0.001
  → 不合格 ×
```

解説　再構成誤差（条件 1）、潜在空間での k 近傍の保存率（条件 2）、デコーダのヤコビ行列の最小特異値（条件 3）を、それぞれ閾値と比較している。結果は条件 1 のみ合格、条件 2 と 3 は不合格であった。

実装上の注意

本編のコードは検証値をハードコードして判定の形だけを示している。実装では、条件 2 は入力空間と潜在空間でそれぞれ k 近傍を求めてその重なりの割合を、条件 3 は関心領域内の多数の代表点でヤコビ行列の特異値分解を行いその最小値を、それぞれ計算する。条件 3 は 1 点だけでなく関心領域全体で満たされる必要があり、代表点の取り方（格子、データ点、ランダム点）を報告に含めるべきである。

2.3　プロジェクト共分散を保存した月次企業状態ベクトル

出典：本編第 2 章

背景　第 2 章「河野遥の提案」。プロジェクト単位の離散的なデータを、月次の連続的な企業状態に変換する方法の提案。

コード

```
# ===== 弧月建設データ再構成 ―― プロジェクト共分散の保存 =====

from sklearn.decomposition import PCA

def construct_monthly_enterprise_state(project_data, monthly_periods):
    """
    プロジェクトレベルのデータを、プロジェクト間の共分散構造を
    保存しながら、月次の企業レベル状態ベクトルに変換する。

    河野遥の提案に基づくアプローチ：
    1. 各月の全進行中プロジェクトの状態ベクトルを収集
    2. PCAで上位k個の共変動パターンを抽出
    3. 月次集約KPI + PCA共変動成分 = 企業状態ベクトル
    """
    enterprise_states = []

    for month in monthly_periods:
        active_projects = project_data[
            (project_data['start_date'] <= month) &
            (project_data['end_date'] >= month)
        ]

        # 1. 月次集約KPI（28指標）
```

```
        monthly_kpi = compute_monthly_kpi(active_projects, month)

        # 2. プロジェクト間の共変動パターン
        if len(active_projects) >= 5:
            project_matrix = active_projects[numeric_columns].values
            pca = PCA(n_components=5)
            pca.fit(project_matrix)
            covariation_scores = pca.transform(project_matrix).mean(axis=0)
        else:
            covariation_scores = np.zeros(5)

        # 3. 結合
        enterprise_state = np.concatenate([monthly_kpi, covariation_scores])
        enterprise_states.append(enterprise_state)

    return np.array(enterprise_states)

X_enterprise = construct_monthly_enterprise_state(
    project_data=all_project_data,
    monthly_periods=pd.date_range('2015-04', '2025-03', freq='M')
)
print(f"企業状態行列：{X_enterprise.shape}")
企業状態行列：(120, 120)
  期間：120ヶ月（10年分）
  次元：120（月次KPI 28 + 共変動5 + 他カテゴリ87 = 120）
```

解説　各月に進行中のプロジェクトを抽出し、(1) 月次集約 KPI と、(2) 進行中のプロジェクト群に PCA を当てて得た上位 5 成分のスコアの平均を連結して、その月の企業状態ベクトルとする。補助文書 01 付録 A で述べたファイバーバンドルの発想を実装したもので、底空間（企業レベル）を連続的に保ちつつ、プロジェクト間の共変動をファイバー側から要約して持ち込んでいる。

実装上の注意

このコードは月ごとに PCA をあらためて当てはめているため、月によって主成分の向きや符号が変わり、5 つの成分の意味が月の間で一貫しない。時系列として連続的な特徴量にするには、全期間のプロジェクト状態をまとめて PCA を一度当てはめ、その固定された主成分で各月を射影するのが望ましい。また、PCA スコアの平均は中心化の仕方によってほぼ 0 になりうるため、平均の代わりに分散や上位スコアなど、プロジェクト群の広がりを表す統計量を併用するとよい。

2.4　命題 1.1 の検証 ― 第 2 回試行

出典：本編第 2 章

背景　企業状態ベクトルに切り替えた後の第 2 回試行。条件 3 は通過したが、条件 2 が 0.86 で未達だった。

コード

```
# ===== 命題1.1検証結果 ―― 弧月建設 第2回試行 =====

mse_reconstruction = 0.0054
print(f"条件1：MSE = {mse_reconstruction:.4f} < 0.01 → 合格 ✓")

knn_preservation_rate = 0.86
print(f"条件2：KNN保存率 = {knn_preservation_rate:.2f} >= 0.90 → 不合格 ×（改善したが未達）")

min_singular_value = 0.0018
print(f"条件3：最小特異値 = {min_singular_value:.4f} >= 0.001 → 合格 ✓")
条件1：MSE = 0.0054 < 0.01 → 合格 ✓
条件2：KNN保存率 = 0.86 >= 0.90 → 不合格 ×（改善したが未達）
条件3：最小特異値 = 0.0018 >= 0.001 → 合格 ✓
```

解説　条件 2（近傍保存率）だけが残ったことは、多様体の局所的な形（ヤコビアン）は改善したが、離れた地理的レジームの間のつながりが不足していることを示す。t-SNE には国内・中東・新興国の 3 つの塊が残っていた。

2.5　技術者異動データの読み込み

出典：本編第 2 章

背景　第 2 章「三浦沙織の洞察」。3 つの地理的レジームをつなぐ「橋」として、技術者の異動履歴を使う場面。

コード

```
# 技術者異動データを遷移信号として活用
transition_events = load_hr_transfers('kogetsu_hr_transfers.csv')
print(f"技術者異動データ：{len(transition_events)}件")
print(f"  国内→中東：{len(transition_events[transition_events['direction']=='JP_to_ME'])}件")
print(f"  中東→国内：{len(transition_events[transition_events['direction']=='ME_to_JP'])}件")
print(f"  国内→東南アジア：{len(transition_events[transition_events['direction']=='JP_to_SEA'])}件")
print(f"  東南アジア→アフリカ：{len(transition_events[transition_events['direction']=='SEA_to_AF'])}件")
技術者異動データ：3,247件
  国内→中東：1,089件
  中東→国内：876件
  国内→東南アジア：534件
  東南アジア→アフリカ：412件
```

解説　SAP の人事モジュールから異動履歴 3,247 件を読み込み、方向別に集計している。この異動イベントが、レジーム間の遷移を表す実データの信号になる。

2.6　制約付き Neural ODE による地理的レジーム間の遷移補間

出典：本編第 2 章

背景　同じく第 2 章。異動データだけでは不足する遷移を、物理的・経営的制約を満たす連続的な軌道で補間する。

コード

```
# ===== レジーム間遷移の補間 ―― 地理的レジーム版 =====
import torchdiffeq

class ConstrainedRegimeTransition(nn.Module):
    """
    地理的レジーム間の遷移をモデル化するNeural ODE。

    制約：
    - 技術者の総数は保存される（保存則）
    - 各地域の最低人員は維持される（安全制約）
    - 資金移動は資金調達能力の範囲内（財務制約）

    ◆ 何をしているか：
      「国内レジーム」と「中東レジーム」の間を、技術者が
      異動する際の経営状態の変化を、物理的制約を満たしながら
      滑らかに補間する。これにより、3つの地理的レジームの
      間に「橋」が架かり、VAEが連続的な多様体を学習できる。
    """
    def __init__(self, latent_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh(),
            nn.Linear(128, latent_dim)
        )

    def forward(self, t, z):
        dz = self.net(z)
        return dz

dynamics = ConstrainedRegimeTransition(latent_dim=12)
synthetic_transitions = torchdiffeq.odeint(
    dynamics, z_start, t_span, method='dopri5'
)
```

解説　潜在空間上のダイナミクス $\dot{z} = f_\theta(z)$ を tanh 活性化のネットワークで表し、torchdiffeq の `odeint`（dopri5）で積分している。tanh を使うのは、Vol.1 で述べられたとおり、曲率

計算に必要な滑らかさを保つためである。

実装上の注意

docstring には技術者総数の保存、最低人員、財務制約が書かれているが、掲載コードのネットワークはこれらの制約を実装していない。実装では、(1) 保存則は出力を射影して総和の変化を 0 にする、(2) 不等式制約は損失関数のペナルティ項やバリア関数で課す、(3) 補間軌道の端点が実データの異動イベントと一致するよう境界条件を損失に含める、などの方法がある。合成した遷移データは実データと区別できる形で保存し、命題 1.1 の検証時にも合成部分の比率を報告すべきである。

2.7　命題 1.1 の検証 ― 第 3 回試行（合格）

出典：本編第 2 章

背景　人材流動データと制約付き補間を加えた第 3 回試行。3 条件すべてが合格した。

コード

```
# ===== 命題1.1検証結果 ―― 弧月建設 第3回試行 =====

mse_reconstruction = 0.0047
print(f"条件1：MSE = {mse_reconstruction:.4f} < 0.01 → 合格 ✓")

knn_preservation_rate = 0.924
print(f"条件2：KNN保存率 = {knn_preservation_rate:.3f} >= 0.90 → 合格 ✓")

min_singular_value = 0.0031
print(f"条件3：最小特異値 = {min_singular_value:.4f} >= 0.001 → 合格 ✓")

print("\n★ 3条件すべて合格 ★")
print("弧月建設 経営環境多様体は、幾何学的解析に適した")
print("有効な12次元微分可能多様体である。")
条件1：MSE = 0.0047 < 0.01 → 合格 ✓
条件2：KNN保存率 = 0.924 >= 0.90 → 合格 ✓
条件3：最小特異値 = 0.0031 >= 0.001 → 合格 ✓

★ 3条件すべて合格 ★
弧月建設 経営環境多様体は、幾何学的解析に適した
有効な12次元微分可能多様体である。
```

解説　条件 2 が 0.924 に改善し、12 次元の経営環境多様体が完成した。ここから先のステップ 3 以降は、この多様体の上で計算される。

第3章　ステップ 3：引き戻し計量の計算

3.1　引き戻し計量と条件数

出典：本編第 3 章

背景　第 3 章「プルバック計量」。多様体の各点で「距離の測り方」を定義し、条件数で異方性を調べる。

コード

```
# ===== ステップ3：プルバック計量の計算 =====

def compute_pullback_metric(decoder, z, epsilon=1e-4):
    """
    多様体上の点zにおけるプルバック計量テンソル g_ij を計算する。

    ◆ 何をしているか：
      VAEのデコーダは、12次元の潜在空間の点を120次元のデータ空間に
      変換する関数 f: R^12 → R^120 である。
      プルバック計量は、この関数のヤコビアン J を使って
      g_ij(z) = J(z)^T @ J(z) として計算される。

    ◆ なぜ重要か：
      プルバック計量 g_ij は12×12の対称行列であり、
      潜在空間の各点で「どの方向に動くと、どれだけ大きな
      変化がデータ空間で起きるか」を記述する。
      固有値が大きい方向は「敏感な方向」、
      固有値が小さい方向は「鈍感な方向」。
    """
    J = np.zeros((decoder.output_dim, len(z)))
    for j in range(len(z)):
        z_plus = z.copy(); z_plus[j] += epsilon
        z_minus = z.copy(); z_minus[j] -= epsilon
        J[:, j] = (decoder(torch.tensor(z_plus)).detach().numpy() -
                   decoder(torch.tensor(z_minus)).detach().numpy()) / (2 * epsilon)
    g = J.T @ J
    return g

sample_points = vae.encode(X_test)
metrics = [compute_pullback_metric(vae.decoder, z) for z in sample_points]

for i, g in enumerate(metrics):
    eigenvalues = np.linalg.eigvalsh(g)
    cn = eigenvalues[-1] / max(eigenvalues[0], 1e-10)
    if cn > 500:
        print(f"警告：点{i}（日付{dates[i]}）の条件数 = {cn:.0f}")
警告：点389（日付2020-04-15）の条件数 = 1847（COVID第一波）
```

```
警告：点547（日付2022-03-01）の条件数 = 923（ウクライナ戦争勃発）
警告：点891（日付2024-10-15）の条件数 = 634
警告：点1142（日付2025-04-07）の条件数 = 478（★サウジNEOM中断★）

計量計算完了：1,200サンプル点
平均条件数：62.8
最大条件数：1847.3
```

解説　デコーダのヤコビ行列 J を中心差分で数値的に求め、$g = J^{\top} J$ を計算している。固有値の最大と最小の比（条件数）が 500 を超える点を警告として出力し、COVID 第一波やサウジ中断の日付が抽出されている。

実装上の注意

中心差分によるヤコビ行列は、刻み幅 ε の選び方で精度が大きく変わり、次元数の 2 倍のデコーダ評価が必要になる。PyTorch では `torch.autograd.functional.jacobian` で厳密なヤコビ行列が得られ、曲率計算で必要になる 2 階微分も自動微分で連鎖的に計算できる（理論論文の第 4.3 節）。また `decoder.output_dim` などは本編の擬似的な属性である。

第4章　ステップ 4：スカラー曲率と測地線距離

4.1　スカラー曲率の計算

出典：本編第 3 章

背景　第 3 章「スカラー曲率」。10 年分のサンプル点で曲率を計算し、不安定領域の上位を抽出する。

コード

```
# ===== ステップ4：スカラー曲率の計算 =====

def compute_scalar_curvature(decoder, z, epsilon=1e-4):
    """
    点zにおけるスカラー曲率Rを計算する。

    ◆ スカラー曲率の意味：
      R > 0（正）：安定。小さな揺れは元に戻る。
      R ≈ 0：中立。
      R < 0（負）：不安定。小さな揺れが自己増幅する。

    ◆ 計算の流れ：
      プルバック計量 → クリストッフェル記号（曲がった空間での方向修正量）
      → リーマン曲率テンソル（曲がり方の完全記述）
      → リッチテンソル → スカラー曲率（1つの数字への要約）
    """
    dim = len(z)
    g = compute_pullback_metric(decoder, z, epsilon)
    g_inv = np.linalg.inv(g)

    christoffel = np.zeros((dim, dim, dim))
    for i in range(dim):
        z_plus_i = z.copy(); z_plus_i[i] += epsilon
        z_minus_i = z.copy(); z_minus_i[i] -= epsilon
        dg_di = (compute_pullback_metric(decoder, z_plus_i) -
                 compute_pullback_metric(decoder, z_minus_i)) / (2 * epsilon)
        for j in range(dim):
            for k in range(dim):
                christoffel[k, i, j] = 0.5 * sum(
                    g_inv[k, l] * (dg_di[l, j] + dg_di[j, l])
                    for l in range(dim))

    R_scalar = compute_ricci_scalar_from_christoffel(christoffel, g_inv, z, epsilon)
    return R_scalar

curvature_map = {}
for i, z in enumerate(sample_points):
```

```
    R = compute_scalar_curvature(vae.decoder, z)
    curvature_map[dates[i]] = R

dangerous = [(d, R) for d, R in curvature_map.items() if R < -5.0]
dangerous.sort(key=lambda x: x[1])

print("弧月建設 多様体上の不安定領域 TOP 10：")
for date, R in dangerous[:10]:
    print(f"  {date}：スカラー曲率 R = {R:.2f}")
```

```
弧月建設 多様体上の不安定領域 TOP 10：
  2020-04-15：R = -58.42  （COVID第一波 —— 全現場一斉停止）
  2022-03-15：R = -41.73  （ウクライナ戦争 —— 鉄鋼価格急騰）
  2020-05-01：R = -35.91  （COVID —— 海外駐在員一斉帰国）
  2023-10-10：R = -29.17  （イスラエル・ハマス紛争 —— 中東リスク急騰）
  2025-04-07：R = -24.83  （★現在★ サウジNEOM中断）
  2022-09-20：R = -19.64  （円安急進 —— 海外資材調達コスト急騰）
  2021-03-25：R = -16.28  （スエズ運河封鎖 —— 資材輸送停滞）
  2024-01-15：R = -13.45  （能登半島地震 —— 復興需要急増と人材不足）
  2025-03-20：R = -11.72  （サイバー探索活動の増加 ★前兆★）
  2019-08-10：R = -8.94   （日韓輸出規制 —— 韓国PJ影響）
```

解説　引き戻し計量 → クリストッフェル記号 → リーマン曲率テンソル → リッチテンソル → スカラー曲率という計算の流れを示している。曲率が -5 を下回る日付を不安定領域として列挙し、COVID 第一波（-58.42）や契約開始時（-24.83）が抽出された。

実装上の注意

掲載コードのクリストッフェル記号は、i 方向の計量の微分だけを使っており、正しい式 $\Gamma^k_{ij} = \frac{1}{2} g^{kl}(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij})$ の第 2 項と第 3 項が欠けている。正しくは、すべての方向 l について計量の偏微分 $\partial_l g_{ij}$ を求めてから組み合わせる必要がある。付録 B.1 に修正版を示す。また、曲率は計量の 2 階微分を含むため、数値差分を入れ子にすると誤差が急速に増える。実運用では自動微分（`create_graph=True`）を用いるべきである。

4.2　測地線距離とユークリッド距離の比較

出典：本編第 3 章

背景　第 3 章。サイバー探索（2025 年 3 月 20 日）とサウジ中断（4 月 7 日）が多様体上で近いかを調べる場面。

コード

```
# サイバー探索（2025-03-20）とサウジ中断（2025-04-07）の測地線距離
z_cyber = vae.encode(X_data['2025-03-20'])
z_saudi = vae.encode(X_data['2025-04-07'])

euclidean_dist = np.linalg.norm(z_cyber - z_saudi)
```

```
geodesic_dist = compute_geodesic_distance(vae.decoder, z_cyber, z_saudi)

print(f"ユークリッド距離：{euclidean_dist:.4f}")
print(f"測地線距離：{geodesic_dist:.4f}")
print(f"比率：{geodesic_dist / euclidean_dist:.2f}")
ユークリッド距離：0.3847
測地線距離：0.4213
比率：1.10
```

解説　2 点間のユークリッド距離と測地線距離を計算し、その比を求めている。比が 1.10 と 1 に近いことから、2 点の間の多様体がほぼ平坦で、両者が隣接していると解釈された。

実装上の注意

`compute_geodesic_distance` は本編では定義されていない。実装では、(1) 2 点を結ぶ離散化した曲線を初期化し、曲線のエネルギー $E = \frac{1}{2}\int g_{ij}\dot{\gamma}^i\dot{\gamma}^j dt$ を勾配法で最小化する方法、(2) 測地線方程式を初期速度について射撃法で解く方法、(3) データ点の近傍グラフ上で引き戻し計量の長さを辺の重みとして最短経路を求める近似、などがある。

第5章　AI エージェントの設計と身体化エージェント

5.1　14 体の AI エージェント・ロスター

出典：本編第 4 章

背景　第 4 章「弧月建設のエージェント・アーキテクチャ」。身体化エージェント（現場系）と非身体化エージェントの役割、入力、出力を定義する。

コード

```
# ===== 弧月建設向け 14体AIエージェント =====

agent_roster = {
    # ──────── 身体化エージェント（現場デジタルツイン層）────────
    'Agent_01_MidEast_Sites': {
        'type': '身体化',
        'domain': '中東建設現場（サウジ・UAE・カタール）',
        'inputs': ['クレーン荷重', 'コンクリート養生温度', '地盤変位',
                   '鉄筋ひずみ', '風速・気温・湿度'],
        'function': '構造安全性監視、品質異常検知、気象リスク評価',
        'world_model': '物理シミュレータ＋異常検知（Isolation Forest + LSTM）',
    },
    'Agent_02_SEA_Sites': {
        'type': '身体化',
        'domain': '東南アジア建設現場（インドネシア・ベトナム）',
        'inputs': ['同上 + 地盤沈下計、地下水位計（軟弱地盤対応）'],
        'function': '軟弱地盤リスク、モンスーン対応、労働安全',
    },
    'Agent_03_Japan_Sites': {
        'type': '身体化',
        'domain': '国内建設現場',
        'inputs': ['地震計、免震装置変位計、トンネル内環境センサー'],
        'function': '地震応答監視、トンネル安全管理',
    },
    'Agent_04_Africa_Sites': {
        'type': '身体化',
        'domain': 'アフリカ建設現場（ケニア・ナイジェリア）',
        'inputs': ['基本環境センサー（IoT展開は限定的）'],
        'function': '限られたセンサーデータからの最大情報抽出',
    },

    # ──────── 非身体化エージェント（経営環境多様体層）────────
    'Agent_05_Market_Sentinel': {
        'type': '非身体化',
        'domain': 'グローバル市場',
        'inputs': ['株価', '為替', 'コモディティ（鉄鋼・セメント・銅）',
                   '建設株価指数', '競合企業の株価'],
```

```
        'function': '市場レジームシフト検知、資材価格予測',
        'world_model': 'Neural ODE + 確率的ボラティリティモデル',
    },
    'Agent_06_Regulatory_Radar': {
        'type': '非身体化',
        'domain': 'グローバル規制',
        'inputs': ['EU CBAM、各国建設基準、環境規制、労働規制'],
        'function': '規制変更の30-90日前予測、コンプライアンスギャップ算出',
    },
    'Agent_07_Geopolitical_Oracle': {
        'type': '非身体化',
        'domain': '地政学リスク',
        'inputs': ['GPR指数', '制裁リスト', 'SC機密脅威インテリジェンス'],
        'function': '地政学リスク進化の監視、制裁予測、ルート評価',
    },
    'Agent_08_Cyber_Guardian': {
        'type': '身体化＋非身体化（ハイブリッド）',
        'domain': 'サイバーセキュリティ',
        'inputs': ['IT-SIEM', 'OT-SIEM（現場IoT）', 'ダークウェブ監視',
                   'SC機密APT情報'],
        'function': 'APT検知、IT/OT横断攻撃の識別、自動隔離トリガー',
        'world_model': 'グラフニューラルネットワーク（ネットワークトポロジー上）',
        'note': '★ 唯一のハイブリッド・エージェント。ITネットワーク（非身体化）と'
                'OTネットワーク（身体化）の両方を監視する。',
    },
    'Agent_09_Project_Portfolio': {
        'type': '非身体化',
        'domain': 'プロジェクト・ポートフォリオ',
        'inputs': ['受注残高', 'EAC乖離率', '技術者配置', 'JVパートナーの信用リスク'],
        'function': 'ポートフォリオ全体のリスク・リターン最適化',
    },
    'Agent_10_Supply_Chain': {
        'type': '非身体化＋身体化',
        'domain': 'サプライチェーン',
        'inputs': ['サプライヤー健全性スコア', '出荷データ', '港湾混雑指数',
                   '現場の資材在庫センサー'],
        'function': '資材調達最適化、代替サプライヤー提案',
    },
    'Agent_11_Financial': {
        'type': '非身体化',
        'domain': '財務・資本',
        'inputs': ['日次P&L', 'キャッシュフロー', 'プロジェクトファイナンス条件'],
        'function': 'G-RAROC日次計算、ヘッジ最適化、資金調達タイミング提案',
    },
    'Agent_12_Talent': {
        'type': '非身体化',
        'domain': '人的資本',
        'inputs': ['技術者稼働率', '海外駐在員配置', '資格保有者DB'],
        'function': '人材配置最適化、キーパーソン離職予測',
    },
    'Agent_13_ESG_Compliance': {
```

```
        'type': '非身体化',
        'domain': 'ESG・環境コンプライアンス',
        'inputs': ['CO2排出量', '廃棄物データ', 'CBAM対象資材使用量'],
        'function': 'カーボンフットプリント予測、CBAM課税額シミュレーション',
    },
    'Agent_14_Orchestrator': {
        'type': 'メタエージェント',
        'domain': 'クロスドメイン統合',
        'inputs': ['他の13体すべてのエージェントの出力'],
        'function': 'エージェント間のリー括弧計算（創発現象検知）、'
                    '矛盾する推奨の解決、CEO向け統合戦略助言の生成',
        'world_model': '経営環境多様体全体 + 全エージェント出力の統合',
    },
}

print(f"配備エージェント総数：{len(agent_roster)}")
print(f"  身体化：4体（現場特化）")
print(f"  非身体化：7体（情報分析特化）")
print(f"  ハイブリッド：2体（IT/OT横断、サプライチェーン）")
print(f"  メタエージェント：1体（オーケストレーター）")
```

解説　各エージェントを辞書で定義し、入力データ、担当する多様体の次元、出力の形式を記述している。Agent_14 はオーケストレーターとして、他の 13 体の出力の間のリー括弧を計算し、矛盾の構造を取締役会に提示する役割を持つ。

実装上の注意

エージェントの定義を設定ファイル（YAML など）として分離し、入力データの権限（どのエージェントが機密情報を読めるか）と出力の承認フロー（どの出力が人間の確認なしに外部へ出てよいか）を明記しておくと、後の形式検証（第 11 章）の対象を明確にできる。

5.2　身体化エージェントのアラート（タワークレーン 8 号機）

出典：本編第 7 章

背景　第 7 章「タワークレーン 8 号機の異常」。アブダビ現場の身体化エージェントが、旋回軸の振動数の倍増を検知した。

コード

```
# Agent_01_UAE（身体化エージェント）アラート
# 日時：2026-05-12T03:17:00+04:00
# 重要度：AMBER → RED（エスカレーション）

alert_embodied = {
    'agent': 'Agent_01_UAE_AbuDhabi',
```

```
    'type': 'embodied',
    'sensor': 'crane_08_vibration_accelerometer',
    'timestamp': '2026-05-12T03:17:00+04:00',
    'finding': {
        'description': 'タワークレーン8号機の旋回軸振動パターンに異常検知',
        'normal_vibration_frequency': '2.3-2.8 Hz（空荷旋回時）',
        'observed_vibration_frequency': '2.3 Hz → 4.7 Hz（15分間で急変）',
        'physical_diagnosis': (
            '旋回軸の固有振動数の倍増は、通常は軸受の損傷または'
            '制御システムの異常を示す。しかし、このクレーンの'
            '軸受は2週間前に点検済みで異常なし。'
            '制御システムの異常の可能性が高い。'
        ),
    },
    'world_model_prediction': (
        'クレーン制御システムが外部から操作された場合、'
        'PLC（プログラマブル・ロジック・コントローラ）の'
        '旋回速度パラメータが変更され、固有振動数の倍増が'
        '生じうる。これはICS/SCADAサイバー攻撃の'
        '物理的シグネチャに一致する。'
    ),
}
```

解説　アラートには、観測値と、物理モデルに基づく推論結果が含まれる。空荷・夜間という条件のもとで振動数が倍増したことを、物理的に説明できる原因（軸受の破損か制御系の異常）に絞り込んでいる。

5.3　非身体化エージェントのアラート（サイバー・ガーディアン）

出典：本編第 7 章

背景　同じ場面で、5 分後に非身体化エージェントがネットワーク側の異常を検知した。

コード

```
# Agent_08（サイバー・ガーディアン）アラート
# 日時：2026-05-12T03:22:00+04:00
# 重要度：AMBER

alert_disembodied = {
    'agent': 'Agent_08_Cyber_Guardian',
    'type': 'disembodied',
    'source': 'SIEM / OT network monitor',
    'timestamp': '2026-05-12T03:22:00+04:00',
    'finding': {
        'description': 'ドバイ支店OTネットワークで異常なトラフィックパターン検知',
        'detail': (
```

```
            'Modbus/TCPプロトコルでのコマンドシーケンスに、'
            '通常の運用パターンには存在しないレジスタ書き込みを検知。'
            '対象：アブダビ現場クレーン制御系PLC。'
            'IPアドレスの起源：TOR出口ノードを経由'
        ),
    },
    'risk_assessment': 'ICS/SCADAへの不正アクセスの可能性 HIGH',
}
```

解説　OT ネットワーク上の非標準の書き込みシーケンスと、既知の攻撃ツールキットとの類似度が報告されている。物理側のアラートとタイムスタンプが近接していることが、両者を突き合わせるきっかけになる。

5.4　身体化 × 非身体化のリー括弧

出典：本編第 7 章

背景　第 7 章。2 つのエージェントの検知をベクトル場とみなし、そのリー括弧で創発効果を測る。

コード

```
# ===== リー括弧：身体化エージェント × 非身体化エージェント =====

def X_physical_anomaly(z):
    """身体化エージェントが検知した物理的異常のベクトル場。"""
    dx = np.zeros_like(z)
    dx[4] = -0.6    # 建設現場の安全性低下
    dx[6] = +0.3    # 工期遅延リスク上昇
    return dx

def X_cyber_intrusion(z):
    """非身体化エージェントが検知したサイバー侵入のベクトル場。"""
    dx = np.zeros_like(z)
    dx[8] = +0.7    # サイバー脅威レベル上昇
    dx[11] = +0.4   # セキュリティ対策コスト増
    return dx

bracket = lie_bracket(X_physical_anomaly, X_cyber_intrusion, z_current)
print(f"リー括弧 [X_physical, X_cyber] の大きさ：{np.linalg.norm(bracket):.4f}")
リー括弧 [X_physical, X_cyber] の大きさ：3.8214

解釈 ―― 身体化 × 非身体化の創発効果：

  身体化エージェントだけなら：「クレーンの故障。修理すれば済む」
  非身体化エージェントだけなら：「サイバー攻撃の痕跡。セキュリティ強化で対処」
```

```
しかしリー括弧が検知した第三の効果：

「建設現場の物理的安全が、サイバー空間から脅かされている」

これは建設業界にとって前例のないリスクカテゴリだ。
クレーンが遠隔操作されれば、300メートルの高さから
数トンの鋼材が落下し、作業員や周辺住民の命が危険にさらされる。

さらに、UAE当局（建設安全監督局）がこの事態を知れば、
弧月建設のUAEでの建設許可が停止される可能性がある。
→ プロジェクト・ファイナンスのコベナンツに二重に抵触
→ 企業全体のレピュテーションリスク

「安全」と「サイバー」のリー括弧は、
「規制」と「金融」の次元に波及する ――
4つのリスクが連鎖する構造。
```

解説　物理的異常と、サイバー侵入をそれぞれ潜在空間のベクトル場として定義し、リー括弧の大きさを計算している。本編では 3.8214 という値が示され、「建設現場の物理的安全がサイバー空間から脅かされている」という第三の効果として解釈された。

実装上の注意

掲載コードのベクトル場は、点 z によらない定数ベクトルを返している。定数ベクトル場どうしのリー括弧は、微分がすべて 0 になるため恒等的に 0 である。リー括弧が 0 でない値を持つためには、ベクトル場の成分が z に依存する（「どの経営状態にいるかによって、事象の押し方が変わる」）ように定義する必要がある。これは GI 理論の出発点である「同じアクションでも、状態空間上の位置によって結果が異なる」という考え方そのものである。付録 B.2 に、状態依存のベクトル場と自動微分によるリー括弧の例を示す。

5.5　物理的ベースライン・チェック

出典：本編第 7 章

背景　第 7 章「弧月の対応」。事件後、三浦沙織の提案で、すべての現場センサーに物理法則との整合性チェックが追加された。

コード

```
# ===== 身体化エージェント強化：物理的ベースライン・チェック =====
# 三浦沙織の提案に基づく

class PhysicsBaselineChecker:
    """
    物理法則に基づくデータ整合性チェック。

    ◆ 何をしているか：
```

```
    センサーの値が物理法則と整合するかを検証する。
    例：風速0 m/sの環境でクレーンの振動が急変した場合、
    外力ではなく内部要因（機械故障またはサイバー攻撃）
    であることを推論できる。

    これはサイバー攻撃に対する「物理的ファイアウォール」
    として機能する ―― データを改ざんしても、
    物理法則に反する値は検知される。
"""

def check_crane_vibration(self, vibration_freq, wind_speed, load_weight):
    """クレーン振動の物理的整合性をチェック"""
    # 空荷・無風での理論的振動数
    theoretical_freq = self.compute_natural_frequency(load_weight)
    # 風による振動の追加成分
    wind_component = 0.1 * wind_speed  # 簡略化モデル
    expected_range = (
        theoretical_freq - 0.5 + wind_component,
        theoretical_freq + 0.5 + wind_component
    )

    if vibration_freq < expected_range[0] or vibration_freq > expected_range[1]:
        return {
            'status': 'ANOMALY',
            'expected': expected_range,
            'observed': vibration_freq,
            'diagnosis': '物理法則と不整合 ―― 機械故障またはサイバー攻撃'
        }
    return {'status': 'NORMAL'}
```

解説　空荷・無風時の理論的な固有振動数に、風による成分を加えた許容範囲を計算し、観測値がその範囲を外れれば異常とする。データが改ざんされても、物理法則と整合しない値は検知できるという「物理的ファイアウォール」の考え方である。

実装上の注意

風による補正を 0.1× 風速とする線形モデルは、本編でも「簡略化モデル」と明記されているとおり説明用である。実運用では、クレーンの構造モデル（吊り荷、ブーム長、旋回角）から固有振動数を計算し、風速・風向・吊り荷の重量などの複数センサーの値の物理的な整合性を同時に確認する。また、センサー値そのものが改ざんされている可能性に備え、独立した系統のセンサー（例：加速度計と荷重計）で相互に検証する設計が望ましい。

第6章　ステップ 5：リー微分とリー括弧

6.1　NEOM 無期限中断のリー微分

出典：本編第 5 章

背景　第 5 章「NEOM 中断の波紋」。NEOM 中断が経営環境の構造をどう変形させるかを計算する。

コード

```
# ===== ステップ5：リー微分 ―― NEOM無期限中断のインパクト =====

def X_neom_suspension(z):
    """
    NEOM無期限中断を表すベクトル場。

    ◆ ベクトル場とは：
      多様体上の各点に、「力の方向と大きさ」を割り当てたもの。
      風の地図のようなもの ―― 各地点で風がどちらにどれだけ
      吹いているかを示す。NEOM中断というイベントが、
      弧月の経営環境の各側面をどの方向にどれだけ
      押し動かすかを定義する。
    """
    dx = np.zeros_like(z)
    dx[0] = -0.7    # 中東売上が大幅減少
    dx[2] = -0.4    # 全社キャッシュフローが圧迫
    dx[5] = +0.3    # 技術者の国内待機コスト増
    dx[7] = +0.5    # 信用リスク上昇（格付機関の注視）
    dx[9] = -0.2    # 受注パイプラインの停滞（他社も様子見）
    return dx

# 現在の経営状態でリー微分を計算
z_current = vae.encode(X_current_state)
L_neom = compute_lie_derivative(vae.decoder, z_current, X_neom_suspension)

eigenvalues_lie = np.linalg.eigvalsh(L_neom)
print("リー微分の固有値スペクトル：")
print(f"  最大固有値：{eigenvalues_lie[-1]:.4f}")
print(f"  最小固有値：{eigenvalues_lie[0]:.4f}")
print(f"  トレース（総変形量）：{np.trace(L_neom):.4f}")
リー微分の固有値スペクトル：
  最大固有値：4.2318
  最小固有値：-3.0547
  トレース（総変形量）：5.8721

解釈：
  NEOM中断のリー微分は、多様体を「コスト」方向に4.2倍膨張させ、
```

```
  「売上」方向に3.1倍収縮させる。
  トレースが正（5.87）—— 経営環境の不確実性が全体的に増大。

  経営陣への翻訳：
  → コスト構造が急速に悪化する（技術者待機費、違約金準備、代替調達費）
  → 売上パイプラインが縮小する（他の発注者も弧月との契約を躊躇し始める）
  → 不確実性が全方位的に拡大する
```

解説　NEOM 中断を潜在空間のベクトル場として定義し、計量のリー微分 $\mathcal{L}_X g$ の固有値とトレースを報告している。本編では、最大固有値 4.23 を「コスト」方向の膨張、最小固有値 -3.05 を「売上」方向の収縮、正のトレースを不確実性の全体的な増大と解釈した。

実装上の注意

リー微分は $(\mathcal{L}_X g)_{ij} = X^k \partial_k g_{ij} + g_{kj}\partial_i X^k + g_{ik}\partial_j X^k$ で計算できる（共変微分を使う形 $\nabla_i X_j + \nabla_j X_i$ と同値）。固有値は「その方向の長さが流れに沿ってどれだけの速さで伸縮するか」を表す変化率であり、本編の「4.2 倍」は経営向けの翻訳である。また、計量に対する固有値は通常の `eigvalsh` ではなく、一般化固有値問題 $\mathcal{L}_X g\, v = \lambda g\, v$ として解くと、座標の取り方によらない量になる。

6.2　リー括弧の計算関数（NEOM 中断 × サイバー探索）

出典：本編第 5 章

背景　第 5 章「リー括弧 — サイバーと中断は共振する」。

コード

```
# ===== リー括弧：NEOM中断 × サイバー探索 =====

def X_cyber_probe(z):
    """サイバー探索活動を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[8] = +0.6    # サイバー脅威レベル上昇
    dx[10] = -0.3   # IoTシステムの信頼性低下
    dx[11] = +0.2   # セキュリティ対策コスト増
    return dx

def lie_bracket(X, Y, z, epsilon=1e-4):
    """
    リー括弧 [X, Y](z) を計算する。

    ◆ リー括弧とは：
      2つのベクトル場XとYの「交差効果」を捉える演算。
      Xが「NEOM中断」、Yが「サイバー探索」のとき、
      [X, Y]は「中断とサイバーが同時に起きたときにだけ
      出現する第三の効果」を表す。
```

```
      数学的には：[X, Y] = dY/dz · X - dX/dz · Y

      直感的には：東に歩いてから北に歩くのと、
      北に歩いてから東に歩くのとで、
      着く場所が違うときに生じる「ズレ」。
      このズレの大きさが、2つの力の相互作用の強さを示す。
    """
    dim = len(z)
    bracket = np.zeros(dim)

    for i in range(dim):
        z_plus = z.copy(); z_plus[i] += epsilon
        z_minus = z.copy(); z_minus[i] -= epsilon

        dY_di = (Y(z_plus) - Y(z_minus)) / (2 * epsilon)
        dX_di = (X(z_plus) - X(z_minus)) / (2 * epsilon)

        bracket += dY_di * X(z)[i] - dX_di * Y(z)[i]

    return bracket

bracket = lie_bracket(X_neom_suspension, X_cyber_probe, z_current)
print(f"リー括弧 [X_neom, X_cyber] の大きさ：{np.linalg.norm(bracket):.4f}")
print(f"方向（正規化）：{bracket / np.linalg.norm(bracket)}")
リー括弧 [X_neom, X_cyber] の大きさ：3.4127
方向（正規化）：[-0.08, -0.34, -0.52, +0.11, -0.22, +0.06, -0.61, +0.15, -0.31, +0.04, -0.18, +0.02]

解釈 ―― 創発現象：
  リー括弧の大きさ 3.41 ―― これは強い創発効果。

  主に潜在次元2（キャッシュフロー）と次元6（信用リスク）の
  負の方向を向いている。

  翻訳：
  → NEOM中断だけなら、売上減少とコスト増。
  → サイバー探索だけなら、セキュリティコスト増。
  → しかし両方が同時に起きると、どちらか単独では生じない
    第三の効果が出現する：

    「プロジェクト・ファイナンスの信用毀損」

    なぜか？ NEOMの中断で弧月の中東事業の信頼性が揺らいでいる最中に、
    サイバー攻撃の懸念が加わると、プロジェクト・ファイナンスの
    貸し手（国際協力銀行、メガバンク、中東の政府系ファンド）が
    「弧月は事業リスクとサイバーリスクの両方を管理できていないのではないか」
    という疑念を持つ。

    融資条件の見直し（コベナンツの厳格化）が発動すれば、
    弧月の資金調達コストが一気に上昇する。
```

これが「第三の効果」—— リー括弧が検知した創発現象。

解説　中心差分で 2 つのベクトル場の微分を求め、$[X,Y]^k = X^i\partial_i Y^k - Y^i\partial_i X^k$ を計算している。式そのものは正しい。

実装上の注意

第 5.3 節で述べたとおり、定義されたベクトル場が定数であれば結果は 0 になる。また、リー括弧はベクトル場の微分のみから定義され、計量を使わない。リー括弧の「大きさ」をユークリッドノルム `np.linalg.norm` で測ると座標の取り方に依存するため、引き戻し計量によるノルム $\sqrt{g_{ij}[X,Y]^i[X,Y]^j}$ で測る方が GI 理論の考え方に合う。

6.3　3 大国の動きと三重リー括弧

出典：本編第 6 章

背景　第 6 章「偶然の裏側」。米国・中国・ロシアの動きをベクトル場として定義し、三重の相互作用を測る。

コード

```
# ===== 3大国の動きをベクトル場として定義 =====

def X_us_tech_control(z):
    """米国の技術移転管理強化を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[1] = -0.3   # 中東プロジェクトでの技術共有が制限
    dx[4] = -0.2   # JVパートナーシップの選択肢が狭まる
    dx[9] = -0.4   # 新規受注の障壁が上昇
    return dx

def X_china_bri_acceleration(z):
    """中国の一帯一路インフラ加速を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[3] = -0.5   # アフリカ・東南アジアでの価格競争が激化
    dx[9] = -0.3   # 弧月の受注パイプラインが浸食される
    dx[11] = -0.2  # 現地政府との関係が希薄化
    return dx

def X_russia_steel_subsidy(z):
    """ロシアの鉄鋼優遇措置を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[6] = +0.4   # 中東での資材調達コスト構造が変化
    dx[3] = -0.2   # ロシア製鉄鋼を使う競合が有利に
    return dx

# 3つのベクトル場の三重リー括弧
```

```
# [X_us, [X_china, X_russia]] —— 3つの力の三重の相互作用
bracket_china_russia = lie_bracket(X_china_bri_acceleration, X_russia_steel_subsidy, z_current)
triple_bracket = lie_bracket(X_us_tech_control, lambda z: bracket_china_russia, z_current)

print(f"三重リー括弧の大きさ：{np.linalg.norm(triple_bracket):.4f}")
三重リー括弧の大きさ：4.8731

解釈：
  三重リー括弧の大きさ 4.87 —— 極めて強い三重の創発効果。

  米国 × 中国 × ロシアの三国の動きが同時に弧月に作用すると、
  いずれか二国の組み合わせからは生じない第三の効果が出現する。

  主にキャッシュフロー（次元2）とプロジェクト・パイプライン
  （次元9）の方向を向いている。

  翻訳：
  → 米国が中東での技術共有を制限 → 弧月のJV戦略が制約される
  → 中国がアフリカ・東南アジアで価格攻勢 → 弧月の代替市場が浸食される
  → ロシアが鉄鋼を安く流す → 弧月のコスト競争力が相対的に低下

  3つが同時に起きると：
  「弧月建設は、中東では米国に制約され、新興国では中国に脅かされ、
  コストではロシアに圧迫される —— 三方向から包囲される」

  これは偶然の一致ではなく、構造的な挟撃である可能性が高い。
```

解説　$[X_{\mathrm{US}}, [X_{\mathrm{China}}, X_{\mathrm{Russia}}]]$ という入れ子のリー括弧を計算している。本編では大きさ 4.87 が報告され、SC クリアランスを起動する判断の根拠になった。

実装上の注意

掲載コードでは、内側のリー括弧を 1 点 z_{current} で評価した定数ベクトルを `lambda z: bracket_china_russia` として外側に渡している。これでは内側のベクトル場の z 依存性が失われ、外側のリー括弧の一方の項が消える。正しくは、内側のリー括弧を「各点 z で評価できる関数」として定義し、その関数を外側に渡す必要がある（付録 B.2）。また、ヤコビ恒等式により 3 通りの入れ子の和は 0 になるため、どの入れ子を計算したかを明示すること。

6.4　6 つの危機の全ペアのリー括弧行列

出典：本編第 10 章

背景　第 10 章「危機 1 と危機 3 は接続している」。6 つの危機のすべての組についてリー括弧を計算し、ネットワークとして整理する。

コード

```
# ===== 全ペアのリー括弧マトリクス =====

events_vectors = {
    'X1_saudi': X_neom_suspension,
    'X2_cyber': X_cyber_probe,
    'X3_indo_reg': X_indo_regulation,
    'X4_uae_attack': X_uae_ics_attack,
    'X5_indo_bribe': X_indo_bribery,
    'X6_steel': X_steel_price_surge,
}

print("リー括弧マトリクス（大きさ）：")
print("（値が大きいほど、2つのイベントの間に強い創発効果がある）")
print()

for name1, X1 in events_vectors.items():
    for name2, X2 in events_vectors.items():
        if name1 < name2:
            bracket = lie_bracket(X1, X2, z_current)
            mag = np.linalg.norm(bracket)
            flag = " ★★★" if mag > 3.0 else (" ★★" if mag > 2.0 else (" ★" if mag > 1.0 else ""))
            print(f"  [{name1}, {name2}] = {mag:.2f}{flag}")
```

```
リー括弧マトリクス（大きさ）：

  [X1_saudi, X2_cyber] = 3.41 ★★★
  [X1_saudi, X3_indo_reg] = 2.87 ★★
  [X1_saudi, X4_uae_attack] = 2.14 ★★
  [X1_saudi, X5_indo_bribe] = 1.93 ★
  [X1_saudi, X6_steel] = 3.72 ★★★
  [X2_cyber, X3_indo_reg] = 0.84
  [X2_cyber, X4_uae_attack] = 4.28 ★★★
  [X2_cyber, X5_indo_bribe] = 0.62
  [X2_cyber, X6_steel] = 1.47 ★
  [X3_indo_reg, X4_uae_attack] = 0.39
  [X3_indo_reg, X5_indo_bribe] = 3.94 ★★★
  [X3_indo_reg, X6_steel] = 2.31 ★★
  [X4_uae_attack, X5_indo_bribe] = 0.71
  [X4_uae_attack, X6_steel] = 1.83 ★
  [X5_indo_bribe, X6_steel] = 2.08 ★★
```

解説　各組の大きさを計算し、3.0 超を ★★★、2.0 超を ★★、1.0 超を ★ として強い創発効果の組を抽出している。サウジ中断がロシア系と中国系の両クラスターと共振するハブであることが、この行列から読み取られた。

実装上の注意

大きさの比較を意味のあるものにするには、各ベクトル場の大きさ（事象の強さ）の尺度を揃えておく必要がある。強い事象どうしのリー括弧は単に大きくなるため、各ベクトル

場を計量ノルムで正規化してからリー括弧を計算し、「相互作用の強さ」と「事象そのものの強さ」を分けて報告するとよい。

第7章　ステップ 8：最適制御

7.1　NEOM 危機の緊急最適制御

出典：本編第 5 章

背景　第 5 章「リー括弧」の後半。180 日の計画期間で、コベナンツ抵触を避ける最適な対応順序を求める。

コード

```
# ===== ステップ8：緊急最適制御 =====

optimal_trajectory = solve_pontryagin(
    initial_state=z_current,
    target_state=z_target,  # 「中東売上安定化、コベナンツ非抵触」
    controls=control_options,
    time_horizon=180,  # 180日の計画期間
    manifold_metric=pullback_metric,
    christoffel_symbols=christoffel,
)

print("最適緊急対応プラン：")
for day, actions in optimal_trajectory.schedule.items():
    print(f"  Day {day}：{actions}")
最適緊急対応プラン：
  Day 0：融資者への先制的開示（NEOM中断の影響と対策を自ら報告）
  Day 3：UAE・カタールの2プロジェクトの加速（代替売上の確保）
  Day 7：サイバーセキュリティの独立監査実施を発表（融資者の懸念を先回り）
  Day 14：国内受注パイプラインの前倒し着手（技術者の待機コスト削減）
  Day 30：NEOM再開に向けたサウジ政府との交渉開始
  Day 60：ポートフォリオ再配分（中東集中度を55%→40%に引き下げ）

総コスト：87億円
コベナンツ抵触確率：68% → 12%に低下
キャッシュフロー改善：月あたり+35億円（Day 60時点）

核心的洞察：
  最適経路は「NEOM再開の交渉」からは始まらない。
  「融資者への先制的開示」から始まる。

  なぜか？ リー括弧が示すように、最大の脅威は
  NEOM中断そのものではなく、中断とサイバーリスクが
  組み合わさって融資者の信頼を毀損すること。
  融資者の信頼を先に確保すれば、NEOM再開の交渉にも
  余裕が生まれる。

  「測地線は、直感が示す方向を必ずしも向いていない」
```

—— リルル先生

解説　ポントリャーギンの最大値原理に基づくソルバー `solve_pontryagin` に、初期状態、目標状態、制御の選択肢、計量とクリストッフェル記号を渡している。結果として、融資者への先制的開示（Day 0）から始まる 6 段階の計画が得られた。

実装上の注意

`solve_pontryagin` は本編では定義されていない。実装では、状態方程式 $\dot{z} = F(z) + uB(z)$ と随伴方程式を前向き・後ろ向きに交互に解く前進後退掃引法や、Neural ODE の随伴法による勾配で制御列を最適化する方法がある（理論論文の第 6.3 節）。本編のように離散的な施策の「順序」を求める問題では、連続最適制御の解を離散的な施策の組み合わせに丸める工程が別途必要になり、その工程は人間のレビュー対象とすべきである。

7.2　インドネシア贈賄危機の最適制御

出典：本編第 8 章

背景　第 8 章「望月常務の判断ミス」。全面対決、協力と並行調査、先制的開示の 3 つの選択肢を比較する。

コード

```
# ===== 最適制御：インドネシア贈賄危機への対応 =====

control_options_indonesia = {
    'A_full_fight': {
        'description': '全面的にKPKと争う。田中の無実を主張',
        'legal_cost': 15,    # 15億円
        'time_to_resolve': 18,  # 18ヶ月
        'project_freeze_probability': 0.75,
    },
    'B_cooperate_and_investigate': {
        'description': 'KPKに全面協力しつつ、社内調査を並行実施',
        'legal_cost': 8,
        'time_to_resolve': 6,
        'project_freeze_probability': 0.25,
    },
    'C_preemptive_disclosure': {
        'description': '弧月から先に社内調査結果を公表し、透明性を示す',
        'legal_cost': 5,
        'time_to_resolve': 4,
        'project_freeze_probability': 0.15,
    },
}

optimal = solve_pontryagin(
```

```
    initial_state=z_current,
    target_state=z_target,
    controls=control_options_indonesia,
    time_horizon=365,
)

print("最適対応：")
print(f"  推奨：{optimal.best_control}")
print(f"  プロジェクト凍結確率：{optimal.freeze_prob:.0%}")
最適対応：
  推奨：C_preemptive_disclosure（先制的開示）
  プロジェクト凍結確率：15%（全面対決なら75%）

核心的洞察：
  多様体上の測地線は「全面対決」を選ばない。
  「先制的開示」―― 弧月自身が社内調査を行い、
  結果を透明に公表すること ―― が、
  プロジェクト凍結リスクを最小化する。

  なぜか？ KPKとの全面対決は、インドネシア政府との
  関係を破壊する。多様体が示すのは、いまの弧月にとって
  インドネシア政府との関係は、田中個人の無罪立証より
  はるかに大きな「資産」であるということ。

  「正しいことをする」と「最善のことをする」は、
  時に異なる方向を指す ―― これが測地線の教えだ。
```

解説　各選択肢に法務コスト、解決までの期間、プロジェクト凍結確率を付与し、最適制御で比較している。結果は先制的開示（凍結確率 15%）が推奨された。

実装上の注意

選択肢ごとの凍結確率などのパラメータは、最適化の結果を大きく左右する入力である。これらが専門家の見積もりなのかデータからの推定なのかを明記し、MC ドロップアウトやシナリオ分析でパラメータの不確実性が結論を覆さないかを確認すること。

第8章　イベント間の幾何学とエージェントのアラート

8.1　NEOM 中断とインドネシア規制変更の測地線距離

出典：本編第 5 章

背景　第 5 章後半。規制レーダーのアラートを受け、2 つの事象の多様体上の近さを調べる。

コード

```
# NEOM中断（2025-10）とインドネシア規制変更（2025-11）の測地線距離
z_neom = vae.encode(X_data['2025-10-01'])
z_indo = vae.encode(X_data['2025-11-15'])

geodesic_dist = compute_geodesic_distance(vae.decoder, z_neom, z_indo)
euclidean_dist = np.linalg.norm(z_neom - z_indo)

print(f"ユークリッド距離：{euclidean_dist:.4f}")
print(f"測地線距離：{geodesic_dist:.4f}")
print(f"比率：{geodesic_dist / euclidean_dist:.2f}")
ユークリッド距離：1.2847
測地線距離：1.3512
比率：1.05
```

解説　第 4.2 節と同じ方法で、測地線距離とユークリッド距離の比（1.05）を計算している。

8.2　5 つのイベントの測地線距離行列

出典：本編第 8 章

背景　第 8 章「逮捕」。5 つの危機の相互距離を行列として整理する。

コード

```
# ===== 多様体上のイベント近接性分析 =====

# 5つの「点」の相互距離マトリクス（測地線距離）
events = {
    'サウジNEOM中断': z_saudi,
    'サイバー探索': z_cyber_probe,
    'インドネシア規制変更': z_indo_reg,
    'UAE ICS攻撃': z_uae_attack,
    'インドネシア贈賄逮捕': z_indo_arrest,
}
```

```
print("イベント間の測地線距離マトリクス：")
print("（距離が小さいほど、多様体上で「近い」＝構造的に類似）")
print()
for name1, z1 in events.items():
    for name2, z2 in events.items():
        if name1 < name2:
            dist = compute_geodesic_distance(vae.decoder, z1, z2)
            flag = " ★近接★" if dist < 0.5 else ""
            print(f"  {name1} ↔ {name2}：{dist:.3f}{flag}")
```

```
イベント間の測地線距離マトリクス：

  サウジNEOM中断 ↔ サイバー探索：0.421 ★近接★
  サウジNEOM中断 ↔ インドネシア規制変更：0.387 ★近接★
  サウジNEOM中断 ↔ UAE ICS攻撃：0.512
  サウジNEOM中断 ↔ インドネシア贈賄逮捕：0.339 ★近接★
  サイバー探索 ↔ インドネシア規制変更：0.723
  サイバー探索 ↔ UAE ICS攻撃：0.298 ★近接★
  サイバー探索 ↔ インドネシア贈賄逮捕：0.651
  インドネシア規制変更 ↔ UAE ICS攻撃：0.814
  インドネシア規制変更 ↔ インドネシア贈賄逮捕：0.187 ★近接★
  UAE ICS攻撃 ↔ インドネシア贈賄逮捕：0.892
```

解説　すべての組の測地線距離を計算し、0.5 未満を「近接」としている。インドネシアの規制変更と贈賄逮捕の距離 0.187 が最も近く、両者が同じ駆動力から生じている可能性を示した。

実装上の注意

「0.5 未満を近接」という閾値は、データの典型的な距離のスケールに依存する。全サンプル点の間の測地線距離の分布を求め、その分位点（例えば下位 5%）を閾値とするなど、閾値の根拠を示すとよい。

8.3　エージェントのアラート形式

本編には、各エージェントのアラートが辞書形式で登場する。ここでは代表として、規制レーダー（第 5 章）と地政学オラクル（第 6 章）のアラートを掲載する。他のアラート（市場センチネル、規制レーダーの CBAM 検知、統合アラート、タレント・コンパス）も同じ形式である。

```
# Agent_06（規制レーダー）アラート
# 日時：2025-11-15
# 重要度：AMBER（警戒）

alert = {
    'agent': 'Agent_06_Regulatory_Radar',
    'timestamp': '2025-11-15T08:23:00Z',
    'type': 'pattern_anomaly',
    'source': 'Indonesia public procurement feed',
```

```
    'finding': (
        'インドネシア公共事業省が、ヌサンタラ新首都建設の'
        '品質監査基準を突然改訂。改訂内容は弧月建設が'
        '使用しているJIS（日本工業規格）ベースの施工基準と'
        '非互換の条項を含む。改訂のパブリックコメント期間は'
        '通常の90日から30日に短縮されている。'
    ),
    'risk_assessment': '弧月のヌサンタラ参画資格に影響する可能性あり',
}
```

```
# Agent_07（地政学オラクル）アラート集約
# 期間：2026-01-10 〜 2026-01-31

alerts_geopolitical = [
    {
        'date': '2026-01-12',
        'source': 'US Congressional Record',
        'finding': '米国上院外交委員会が、中東における「同盟国の技術移転管理」'
                   'に関する公聴会を開催。日本企業の名前は出ていないが、'
                   '「特定のインフラプロジェクトにおける技術流出リスク」が議題',
    },
    {
        'date': '2026-01-18',
        'source': 'MOFCOM（中国商務部）',
        'finding': '中国が「一帯一路インフラ建設加速化プログラム」を発表。'
                   '重点地域にインドネシア・ケニア・ナイジェリアを指定。'
                   '弧月建設がプロジェクトを進行中の3カ国すべてが含まれる',
    },
    {
        'date': '2026-01-25',
        'source': 'ロシア国営メディア RT',
        'finding': 'ロシアが中東の鉄鋼輸出に関する新たな優遇措置を発表。'
                   '「友好国」への鉄鋼輸出価格を20%割引。サウジ、UAE、'
                   'カタールが「友好国」リストに含まれる',
    },
]

print(f"地政学アラート：{len(alerts_geopolitical)}件")
for a in alerts_geopolitical:
    print(f"  {a['date']}：{a['source']}")
```

実装上の注意

アラートには、検知した事実（finding）と、エージェントの解釈を分けて記録しておくことが望ましい。事実と解釈が混ざると、後から人間が判断の根拠を検証しにくくなる。また、機密情報に基づくアラートは、出力先（どの取締役に共有してよいか）の分類をアラート自体に持たせておく必要がある。

第9章　ステップ 7：DML による因果分解

9.1　鉄鋼価格上昇の因果分解

出典：本編第 9 章

背景　第 9 章「6 つ目の点」。鉄鋼価格の 40% の上昇のうち、台湾海峡の緊張で説明できる部分を推定する。

コード

```
# ===== DML因果分析：鉄鋼価格上昇の因果分解 =====

# DML（ダブル機械学習）とは：
# 機械学習を使って「交絡因子」（他の要因）の影響を除去し、
# 特定の原因が結果にどれだけ影響しているかを推定する手法。
# 通常の回帰分析よりも、因果効果の推定が正確。

from econml.dml import LinearDML

# 処置変数（Treatment）：台湾海峡緊張指数
# 結果変数（Outcome）：鉄鋼スポット価格
# 交絡因子（Confounders）：世界GDP成長率、建設需要指数、
#    中国鉄鋼生産量、エネルギー価格、為替レート

dml = LinearDML(model_y='auto', model_t='auto')
dml.fit(Y=steel_price, T=taiwan_tension_index, X=other_factors, W=confounders)

ate_taiwan = dml.ate()
print(f"台湾海峡緊張の鉄鋼価格への因果効果：{ate_taiwan:.1f}%")
print(f"観測された価格上昇：40%")
print(f"台湾で説明される分：{ate_taiwan:.1f}%")
print(f"説明されない分：{40 - ate_taiwan:.1f}%")
台湾海峡緊張の鉄鋼価格への因果効果：12.3%
観測された価格上昇：40%
台湾で説明される分：12.3%
説明されない分：27.7%

追加分析 ―― 説明されない27.7%の推定要因：
  ロシアの中東向け鉄鋼優遇措置：+8.2%（推定因果効果）
    → ロシアが中東市場に安い鉄鋼を流し込むことで、
      中東以外の市場から鉄鋼が引き上げられ、
      アジア・欧州の鉄鋼価格が上昇
  中国の鉄鋼輸出規制強化：+11.4%
    → 中国が自国のインフラ建設のために鉄鋼輸出を抑制
  残余（説明不能）：+8.1%
```

解説　EconML の `LinearDML` で、処置変数を台湾海峡緊張指数、結果変数を鉄鋼スポット価格として平均処置効果を推定している。結果は 12.3% で、残り 27.7% は他の原因を探す必要があると示された。

実装上の注意

`LinearDML` の `ate()` が返すのは、処置変数を 1 単位変えたときの結果変数の変化である。「価格上昇率に対する寄与（%）」として読むには、結果変数を対数価格の変化率とし、処置変数を実際に観測された緊張指数の変化幅で評価する必要がある。また、複数の原因について別々に推定した効果を足し合わせて全体と比較する場合、各推定の識別条件（交絡因子の選び方）が整合しているかを確認すること。DML の推定値には信頼区間（`ate_interval()`）を必ず付ける。

第10章　幾何学的拡張

10.1　EU CBAM のベクトル場と 7 つ目の点

出典：本編第 13 章

背景　第 13 章「7 つ目の点」。EU CBAM の適用範囲拡大をベクトル場として定義し、発表時点の曲率を計算する。

コード

```
def X_eu_cbam(z):
    """EU CBAM建設サービス拡大を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[2] = -0.5    # 欧州売上の利益率が低下
    dx[6] = +0.6    # 規制コンプライアンスコスト急増
    dx[9] = -0.3    # 欧州での新規受注パイプラインが停滞
    dx[11] = +0.2   # ESG関連の投資負担増
    return dx

# 多様体に7つ目の点を刻む
z_cbam = vae.encode(X_data['2027-07-15'])
curvature_cbam = compute_scalar_curvature(vae.decoder, z_cbam)
print(f"CBAM発表時点のスカラー曲率：{curvature_cbam:.2f}")
CBAM発表時点のスカラー曲率：-28.41

※ サウジNEOM中断時（-24.83）より深い谷。
  弧月の経営環境は2年間で最も不安定な状態に到達。
```

解説　発表時点のスカラー曲率は -28.41 で、契約開始時（-24.83）よりも深い不安定領域に入ったことが示された。

10.2　パーシステント・ホモロジー

出典：本編第 13 章

背景　第 13 章「パーシステント・ホモロジー ― 7 つの点が 1 つの形を結ぶ」。

コード

```
# ===== パーシステント・ホモロジー：7つの危機の位相解析 =====

from ripser import ripser
from persim import plot_diagrams
import numpy as np
```

```
# 7つのイベントの多様体上の座標（12次元潜在空間）
crisis_points = np.array([
    z_saudi,         # ① サウジNEOM中断
    z_cyber_probe,   # ② サイバー探索
    z_indo_reg,      # ③ インドネシア規制変更
    z_uae_attack,    # ④ UAE ICS攻撃
    z_indo_bribe,    # ⑤ インドネシア贈賄
    z_steel_surge,   # ⑥ 鉄鋼価格操作
    z_cbam,          # ⑦ EU CBAM
])

# パーシステント・ホモロジーを計算
result = ripser(crisis_points, maxdim=2)

print("パーシステント・ホモロジー結果：")
print()

# H0（連結成分）
h0 = result['dgms'][0]
print(f"H0（連結成分）：{len(h0)}個の成分")
print(f"  最も持続する2成分の持続性：")
for i, (birth, death) in enumerate(sorted(h0, key=lambda x: -(x[1]-x[0]))[:2]):
    print(f"    成分{i+1}：誕生={birth:.3f}, 消滅={death:.3f}, 持続性={death-birth:.3f}")

# H1（ループ）
h1 = result['dgms'][1]
print(f"\nH1（ループ）：{len(h1)}個のループ")
for i, (birth, death) in enumerate(h1):
    persistence = death - birth
    print(f"  ループ{i+1}：誕生={birth:.3f}, 消滅={death:.3f}, 持続性={persistence:.3f}")

# H2（空洞）
if len(result['dgms']) > 2:
    h2 = result['dgms'][2]
    print(f"\nH2（空洞）：{len(h2)}個")
```

```
パーシステント・ホモロジー結果：

H0（連結成分）：2個の持続的な成分
  最も持続する2成分の持続性：
    成分1：誕生=0.000, 消滅=∞, 持続性=∞（全体の連結成分）
    成分2：誕生=0.000, 消滅=0.812, 持続性=0.812
    → 7つの点は最終的に1つの連結成分に統合される。
      しかし距離0.812までは2つのグループに分かれている。
      グループA：①サウジ、②サイバー、④UAE攻撃、⑥鉄鋼（ロシア系+米国系）
      グループB：③インドネシア規制、⑤インドネシア贈賄（中国系）
      ⑦EU CBAMは距離0.812で両グループをつなぐ「橋」

H1（ループ）：2個のループ
  ループ1：誕生=0.298, 消滅=1.147, 持続性=0.849 ★★★持続的★★★
    → ①サウジ → ②サイバー → ④UAE攻撃 → ⑥鉄鋼 → ①サウジ
```

```
      の循環構造。ロシアの作戦が米国の圧力を利用して
      弧月を繰り返し攻撃する「フィードバックループ」

  ループ2：誕生=0.187，消滅=0.934，持続性=0.747 ★★持続的★★
    → ③インドネシア規制 → ⑤インドネシア贈賄 → ⑦EU CBAM
      → ③インドネシア規制 の循環構造。
      規制環境を操作して弧月を市場から排除する
      「規制包囲ループ」

H2（空洞）：0個
  → 7つの点は中空の球のような構造は形成していない。
```

解説　ripser で 7 つの危機の点群のパーシステンス図を計算し、H0、H1、H2 の各成分の誕生と消滅を出力している。本編では H1（ループ）が 2 個検出され、「攻撃ループ」と「規制包囲ループ」と解釈された。

実装上の注意

掲載コードは 12 次元の潜在座標をそのまま ripser に渡しているため、点間の距離はユークリッド距離で計算される。GI 理論の考え方に従うなら、第 8.2 節の測地線距離行列を `ripser(D, distance_matrix=True, maxdim=2)` として渡すべきである（付録 B.3）。また、7 点という少数の点では、1 点の位置の誤差でループが現れたり消えたりしうる。ブートストラップや点の位置の摂動で、検出されたループの持続性が安定しているかを確認すること。

10.3　多人数ゲーム理論

出典：本編第 16 章

背景　第 16 章「敵の戦略空間」。米国・中国・ロシア・弧月の 4 者の戦略と利得を定義し、ナッシュ均衡を求める。

コード

```
# ===== 第23章：多人数ゲーム理論 —— 経営環境多様体上のナッシュ均衡 =====

# プレイヤーと戦略空間の定義

players = {
    'US': {
        'objective': '中東における技術移転リスクの最小化',
        'strategies': [
            'S1_maintain_pressure',   # サウジへの圧力を維持
            'S2_selective_exemption', # 弧月に個別の免除を付与
            'S3_escalate',            # 全面的な技術移転規制
        ],
    },
    'China': {
```

```
        'objective': '一帯一路の建設市場シェア最大化',
        'strategies': [
            'S1_price_war',          # 価格攻勢で弧月を排除
            'S2_regulatory_capture', # 規制変更で弧月を制度的に排除
            'S3_covert_ops',         # スリヤント型の内部工作
        ],
    },
    'Russia': {
        'objective': 'NATO諸国の中東インフラ影響力の低減',
        'strategies': [
            'S1_cyber_erosion',      # 信頼の侵食（サイバー攻撃）
            'S2_commodity_manipulation', # 鉄鋼等の価格操作
            'S3_combined',           # サイバー＋価格操作の同時展開
        ],
    },
    'Kogetsu': {
        'objective': '企業価値の最大化と事業継続性の確保',
        'strategies': [
            'S1_status_quo',         # 現状維持（個別対応）
            'S2_portfolio_surgery',  # ポートフォリオの位相変更
            'S3_alliance',           # 地政学的同盟の構築（日豪米等）
            'S4_surgery_plus_alliance', # 手術＋同盟の複合戦略
        ],
    },
}

# ナッシュ均衡の計算
# 各プレイヤーが他のプレイヤーの戦略を所与として、
# 自分の戦略を変更するインセンティブがない状態を求める

nash_equilibria = compute_nash_equilibria(
    players=players,
    payoff_functions=payoff_matrix,  # 多様体上のリー微分から導出
    manifold=vae_manifold,
)

print("ナッシュ均衡分析結果：")
for eq in nash_equilibria:
    print(f"\n均衡{eq.id}：")
    for player, strategy in eq.strategies.items():
        print(f"  {player}：{strategy}")
    print(f"  弧月の企業価値（5年後）：{eq.kogetsu_value:.0f}億円")
```

```
ナッシュ均衡分析結果：

均衡1（最も蓋然性が高い）：
  US：S1_maintain_pressure（サウジへの圧力維持）
  China：S2_regulatory_capture（規制による排除を継続）
  Russia：S3_combined（サイバー＋価格操作の同時展開）
  Kogetsu：S4_surgery_plus_alliance（手術＋同盟）
  弧月の企業価値（5年後）：18,200億円（現在比 +21.3%）
```

```
均衡2（弧月が現状維持の場合）：
  US：S1_maintain_pressure
  China：S1_price_war → S2_regulatory_capture（エスカレーション）
  Russia：S3_combined
  Kogetsu：S1_status_quo（個別対応）
  弧月の企業価値（5年後）：9,400億円（現在比 -37.3%）

核心的洞察：
  弧月が「現状維持」を選べば、3つの国家はそれぞれの戦略を
  エスカレートさせる（弧月が抵抗しないなら、もっと押すのが合理的）。
  5年で企業価値が37%毀損。

  しかし弧月が「手術＋同盟」を選べば、3つの国家にとっての
  "弧月を攻撃するコスト"が上昇する。
  特に「同盟」―― 日豪米の戦略的パートナーシップ ―― は、
  米国の利害を弧月の防衛側に転換する効果がある。

  米国が「同盟国としての弧月を支援する」側に回れば、
  ゲームの構造が根本的に変わる。
  3 vs 1（米中口 vs 弧月）が 2 vs 2（中口 vs 日米）に転換する。
```

解説　各プレイヤーの目的と戦略の選択肢を辞書で定義し、利得を経営環境多様体上の位置の変化として評価している。本編では、弧月が手術と同盟を選ぶ均衡と、現状維持を選ぶ均衡が比較された。

実装上の注意

4 者がそれぞれ 3〜4 個の戦略を持つ有限ゲームでは、純粋戦略のナッシュ均衡が存在しないこともある。その場合は混合戦略の均衡を求める必要がある。nashpy は 2 人ゲーム向けのため、多人数ゲームには Gambit などのソルバーを使う。また、各国の利得関数は推定に大きく依存するため、利得の仮定を変えても均衡が変わらないかを感度分析で確認すること。

10.4　手術理論によるポートフォリオの位相変換

出典：本編第 18 章

背景　第 18 章「ポートフォリオの位相を変える」。

コード

```
# ===== 第13章：手術理論の適用 ―― ポートフォリオの位相変換 =====

# 現在のポートフォリオの位相：ハブ・アンド・スポーク
# ハブ：中東（売上25%）
# スポーク：国内（35%）、東南アジア（15%）、アフリカ（5%）、
#           欧米（10%）、再エネ（10%）
```

```
# 問題：パーシステント・ホモロジーが検出した2つのループは、
# いずれも中東ハブを経由している。ハブを排除すれば、
# ループの接続が断たれる。

surgery_plan = {
    'cut': [
        'サウジNEOMプラント ―― 無期限中断のまま撤退判断',
        'アフリカ低収益プロジェクト3件 ―― 中国との価格戦争が不採算化',
    ],
    'glue': [
        'オーストラリア再エネインフラ ―― 日豪QUADフレームワーク活用',
        'カナダ・ブリティッシュコロンビア州LNG基地建設',
        '日本国内データセンター建設 ―― 半導体戦略と連動',
        '米国テキサス州水素製造プラント ―― IRA（インフレ削減法）補助金活用',
    ],
    'cobordism_check': (
        '切除境界（中東＋アフリカ削減分：約4,500億円の売上）と'
        '接着境界（豪＋加＋日＋米の新規売上：約4,200億円見込み）が'
        '±300億円の範囲で一致 → コボルディズム条件は概ね充足'
    ),
}

print("手術計画：")
print(f"  切除：{len(surgery_plan['cut'])}件")
print(f"  接着：{len(surgery_plan['glue'])}件")
print(f"  コボルディズム条件：概ね充足")
print()
print("手術後のポートフォリオ：")
print("  国内：40%（+5%）")
print("  中東：12%（-13%）―― UAEとカタールのみ維持")
print("  東南アジア：14%（-1%）")
print("  アフリカ：2%（-3%）")
print("  豪州：8%（新規）")
print("  北米：12%（+7%）")
print("  欧州：5%（維持）")
print("  再エネ：7%（-3%、構成変更）")
print()
print("位相変換の効果：")
print("  パーシステント・ホモロジー ループ1：消滅（サウジハブの除去）")
print("  パーシステント・ホモロジー ループ2：弱体化（アフリカ撤退により中国系接点が減少）")
print("  地政学リスク集中度指数：0.78 → 0.34")
```

解説　切除（サウジ撤退、アフリカの低収益案件）と接着（豪州、カナダ、日本、米国の新規案件）の計画を定義し、切除境界と接着境界の売上規模の一致を「コボルディズム条件」として確認している。手術後に、パーシステント・ホモロジーのループ 1 が消滅することが示された。

実装上の注意

このコードの手術計画は、数学的な手術理論の計算ではなく、ポートフォリオ再編の計画を手術理論の言葉で整理したものである。実装で検証可能な部分は、(1) 手術後のポートフォリオで多様体上の位置を再計算し、(2) 危機の点群のパーシステント・ホモロジーを再計算してループが消えるかを確かめる、という工程である。

10.5　シンプレクティック積分による 7 年間のシミュレーション

出典：本編第 19 章

背景　第 19 章「7 年間の航路」。手術あり・なしの 2 つのシナリオについて 84 ヶ月の軌道を計算する。

コード

```
# ===== 第15章：シンプレクティック積分 ―― 7年間のシミュレーション =====

def stormer_verlet_step(z, p, dt, hamiltonian_grad):
    """
    シュテルマー＝ヴェルレ法の1ステップ。

    ◆ 何をしているか：
      経営状態(z)と経営の"勢い"(p)を、ハミルトニアン力学に従って
      時間発展させる。通常の数値積分と異なり、エネルギー
      （ハミルトニアン H = 経営の総ポテンシャル）が保存される。

    ◆ なぜ重要か：
      7年間のシミュレーションで通常の積分法を使うと、
      小さな誤差が雪だるま式に蓄積し、5年後の予測は
      まったく無意味になる。シンプレクティック積分は
      誤差の蓄積を防ぎ、長期予測の信頼性を保証する。
    """
    # 半ステップで運動量を更新
    p_half = p - 0.5 * dt * hamiltonian_grad(z, p, 'dH_dz')
    # 全ステップで位置を更新
    z_new = z + dt * hamiltonian_grad(z, p_half, 'dH_dp')
    # 残り半ステップで運動量を更新
    p_new = p_half - 0.5 * dt * hamiltonian_grad(z_new, p_half, 'dH_dz')
    return z_new, p_new

# 7年間のシミュレーション（手術後のポートフォリオ）
trajectory_surgery = simulate_symplectic(
    z_initial=z_post_surgery,
    time_horizon=84,  # 84ヶ月 = 7年
    dt=1.0,  # 月次ステップ
    method='stormer_verlet',
)
```

```
# 比較：手術なしの場合
trajectory_no_surgery = simulate_symplectic(
    z_initial=z_current,
    time_horizon=84,
    dt=1.0,
    method='stormer_verlet',
)

# ハミルトニアンの保存性チェック
H_initial = compute_hamiltonian(z_post_surgery, p_initial)
H_final = compute_hamiltonian(trajectory_surgery[-1], p_final)
print(f"ハミルトニアン保存性：ΔH/H = {abs(H_final - H_initial) / H_initial:.2e}")
7年間シンプレクティック・シミュレーション結果：

ハミルトニアン保存性：ΔH/H = 2.87×10^-8
  → 7年間で10億分の2.87のエネルギー誤差。極めて高い精度。

【手術あり（推奨シナリオ）】
  企業価値（2028年3月）：1兆5,000億円（現在）
  企業価値（2031年3月）：1兆8,900億円（+26.0%）
  企業価値（2035年3月）：2兆3,200億円（+54.7%）
  営業利益率：8.4% → 12.7%
  地政学リスク集中度：0.78 → 0.31（安全域）

【手術なし（現状維持シナリオ）】
  企業価値（2028年3月）：1兆5,000億円（現在）
  企業価値（2031年3月）：1兆1,200億円（-25.3%）
  企業価値（2035年3月）：8,400億円（-44.0%）
  営業利益率：8.4% → 3.8%
  地政学リスク集中度：0.78 → 0.91（危険域）

差分：
  7年後の企業価値差：2兆3,200億 - 8,400億 = 1兆4,800億円
  手術コスト：450億円
  ROI：3,189%
```

解説　シュテルマー＝ヴェルレ法（速度ヴェルレ法）の 1 ステップを、運動量の半ステップ更新、位置の全ステップ更新、運動量の半ステップ更新の 3 段で実装している。

実装上の注意

この陽的な形のシュテルマー＝ヴェルレ法がシンプレクティックになるのは、ハミルトニアンが $H(z,p) = T(p) + V(z)$ のように位置と運動量で分離できる場合である。リーマン多様体上の運動では運動エネルギー $\frac{1}{2} g^{ij}(z) p_i p_j$ が位置 z に依存し、ハミルトニアンは分離できない。この場合は、陰的な一般化リープフロッグ法（暗黙的な半ステップを反復で解く方法）を使う必要がある。付録 B.4 に考え方を示す。

10.6　同盟構築後の多様体の更新

出典：本編第 20 章

背景　第 20 章「ゲームの構造転換」。同盟構築後の経営状態で曲率を再計算する。

コード

```
# 同盟構築後の多様体更新
z_post_alliance = vae.encode(X_data_post_alliance)
curvature_post = compute_scalar_curvature(vae.decoder, z_post_alliance)
print(f"同盟構築後のスカラー曲率：{curvature_post:.2f}")
同盟構築後のスカラー曲率：-4.12

（比較）
  案件開始時（2025年4月）：-24.83
  CBAM発表時（2027年7月）：-28.41（最悪期）
  手術直後（2028年3月）：-11.37
  同盟構築後（2028年5月）：-4.12

  → 3年間で最も安定した水準。
    負の曲率はまだ残るが、「崖の縁」から
    「なだらかな坂道」まで回復。
```

解説　同盟構築後のスカラー曲率は -4.12 で、契約開始時の -24.83、最悪期の -28.41、手術直後の -11.37 と比べ、3 年間で最も安定した水準になった。

10.7　層理論による内部情報漏洩の検知

出典：本編第 21 章

背景　第 21 章「層理論が暴く最後の脆弱性」。タレント・コンパスのアラートを受け、組織構造上の層で整合性を検証する。

コード

```
# ===== 第18章：層理論による内部整合性チェック =====

def detect_information_leak_via_sheaf(employee_data, access_logs, email_logs):
    """
    層のラプラシアンで内部情報漏洩を検知する。

    モデル：組織構造のグラフ上に層を定義。
    各ノード（部署/個人）にアクセスできるデータの範囲を「層データ」として割り当て。
    メール・ファイル転送パターンが、アクセス権限と整合するかを検証。
```

```
    """
    sheaf = construct_org_sheaf(employee_data, access_logs)
    L = sheaf.laplacian()

    for employee in email_logs:
        x = sheaf.get_data_vector(employee)
        inconsistency = np.linalg.norm(L @ x)
        if inconsistency > threshold:
            print(f"不整合検知：{employee.name}, スコア={inconsistency:.2f}")

# 実行結果
# 不整合検知：大島真由美, スコア=6.31
# → 大島のメール転送パターンは、彼女のアクセス権限の範囲と
#    「整合しない」―― つまり、彼女がアクセスできるはずのない
#    データを転送している痕跡がある
```

解説　組織構造のグラフ上に層を構成し、各個人のデータベクトルに層ラプラシアンを作用させ、その大きさを不整合スコアとしている。ある職員のスコア 6.31 が閾値を超え、転送パターンがアクセス権限と整合しないことが検出された。

実装上の注意

層ラプラシアンを定義するには、各ノード（部署・個人）のデータ空間（ストーク）と、各辺の制限写像を具体的に決める必要がある。例えば「上司のアクセス範囲から部下のアクセス範囲への射影」を制限写像とすれば、層ラプラシアンの値は「隣接するノードの間で、本来一致すべきデータが一致していない度合い」を表す。閾値は正常期間のスコアの分布から決め、誤検知時の調査手順（誰が確認し、本人の権利をどう守るか）をあらかじめ定めておくことが重要である。

10.8　座標系の変換（本編の呼称：ゲージ変換）

出典：本編第 22 章

背景　第 22 章「反撃」。流出した多様体分析を無効化するため、潜在空間を新しい座標系に移す。

コード

```
# ===== ゲージ変換：座標系の回転 =====

# 漏洩した座標系を無効化
# 新しいランダム直交行列Rで潜在空間を回転
R = random_orthogonal_matrix(dim=12, seed=secure_random_seed)

# 新座標系でVAEを再パラメータ化
vae_new = apply_gauge_transformation(vae, R)
```

```
# 検証：新座標系での多様体の幾何学的性質は不変
#  （曲率、測地線距離などはゲージ不変量）
curvature_old = compute_scalar_curvature(vae.decoder, z_current)
curvature_new = compute_scalar_curvature(vae_new.decoder, R @ z_current)
print(f"旧座標系の曲率：{curvature_old:.4f}")
print(f"新座標系の曲率：{curvature_new:.4f}")
print(f"差分：{abs(curvature_old - curvature_new):.6f}（数値誤差の範囲内）")
旧座標系の曲率：-4.1237
新座標系の曲率：-4.1238
差分：0.000014（数値誤差の範囲内）

→ 多様体の幾何学的性質（曲率、測地線距離）は完全に保存。
  しかし潜在空間の座標値は完全に変わった。
  漏洩したデータは無効化された。
```

解説　ランダムな直交行列 R で潜在空間を回転し、新しい座標系でデコーダを再パラメータ化する。変換前後でスカラー曲率が一致すること（-4.1237 と -4.1238）を確認し、幾何学的な性質が保たれたまま座標値だけが変わったことを示している。

実装上の注意

新しいデコーダは $f_{\text{new}}(w) = f(R^\top w)$ と定義する必要がある（$w = Rz$）。この定義なら引き戻し計量は $g_{\text{new}} = R\, g\, R^\top$ となり、曲率などの幾何学的な量は厳密に保存される。一方、回転は線形変換なので、流出した古い座標と新しい座標の対応が 1 組でも知られると R が推定されうる。座標系の変換は、アクセス権限の見直しや鍵の更新と組み合わせて使うべきである。

第11章　形式検証と ZKML

11.1　ZKML 証明の生成

出典：本編第 23 章

背景　第 23 章「最終証明」。モデルの重みや機密データを開示せずに、計算が正しいことを取締役会に証明する。

コード

```
# ===== ZKML証明の生成 =====

print("ZKML証明生成：")
print("  回路サイズ：3.8M ゲート（弧月の多様体は綾鳥より複雑）")
print("  証明生成時間：1時間23分")
print("  証明サイズ：2.1 MB")
print("  検証時間：1.2秒")
print("  結果：証明有効 ✓")
print()
print("弧月の取締役会は以下を検証できる：")
print("  1. 多様体が120指標のデータから正しく構築されたこと")
print("  2. パーシステント・ホモロジーの計算が正しいこと")
print("  3. 手術理論の適用が数学的に有効であること")
print("  4. シンプレクティック積分の7年予測が計算的に正しいこと")
print("  ── すべて、モデルの重みやSC機密データを見ることなく")
```

解説　回路サイズ、証明生成時間、証明サイズ、検証時間を出力し、取締役会が検証できる内容を列挙している。

実装上の注意

ZKML は計算を算術回路に変換して証明を作るため、浮動小数点演算は固定小数点に量子化する必要があり、その誤差が結果に影響しないかを確認すること。また、ZKML が保証するのは「約束したモデルが正しく実行されたこと」であり、入力データの選び方の公正性は保証しない。理論論文の 3 つの防御層（ZKML、訓練ログのコミットメント、第三者監査）を組み合わせて運用する。形式検証（Coq / Z3 / Agda）については本編にコードが掲載されていないため、1 作目の補助文書 03 の第 13.1 節を参照されたい。

付録 A　推奨ライブラリ一覧

分野	ライブラリ	本書での用途
数値計算	NumPy, SciPy, pandas	行列計算、固有値、データ処理
深層学習	PyTorch	VAE、自動微分によるヤコビ行列・曲率
Neural ODE	torchdiffeq	制約付き遷移補間、連続時間ダイナミクス
機械学習	scikit-learn	PCA、近傍探索、固有次元推定
因果推論	EconML	DML による因果効果の推定
説明可能性	SHAP	要因分解
位相的データ解析	ripser, persim, giotto-tda	パーシステント・ホモロジー
ゲーム理論	Gambit, nashpy	ナッシュ均衡（nashpy は 2 人ゲーム向け）
層理論	自作実装、または PySheaf 等	層ラプラシアン
形式検証	Coq, Lean 4, Agda, Z3 (z3-solver)	安全性制約の証明
ZKML	EZKL など	証明の生成と検証

インストールの例：

```
pip install numpy scipy pandas scikit-learn torch torchdiffeq
pip install econml shap ripser persim nashpy z3-solver
```

付録 B　補足の参照実装

本付録のコードは、本文の「実装上の注意」で述べた点を示すための最小限の例であり、本編の数値を再現するものではない。

B.1　自動微分によるクリストッフェル記号

```
import torch

def metric(decoder, z):
    J = torch.autograd.functional.jacobian(decoder, z, create_graph=True)  # (D, d)
    return J.T @ J                                                          # (d, d)

def christoffel(decoder, z):
    z = z.detach().requires_grad_(True)
    g = metric(decoder, z)
    g_inv = torch.linalg.inv(g)
    d = z.shape[0]
    # dg[l, i, j] = ∂_l g_ij （すべての方向 l について計算する）
    dg = torch.stack([torch.autograd.grad(g[i, j], z, create_graph=True)[0]
                      for i in range(d) for j in range(d)], dim=1).reshape(d, d, d)
    A = dg                    # A[i, j, l] = ∂_i g_jl
    B = dg.permute(1, 0, 2)   # B[i, j, l] = ∂_j g_il
    C = dg.permute(1, 2, 0)   # C[i, j, l] = ∂_l g_ij
    # Γ^k_ij = 1/2 g^{kl} (∂_i g_jl + ∂_j g_il − ∂_l g_ij)
    return 0.5 * torch.einsum('kl,ijl->kij', g_inv, A + B - C)
```

要点は「$\partial_l g_{ij}$ をすべての l について求め、$\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij}$ を組み立てる」ことである。実装後は、平面の極座標や球面など、クリストッフェル記号が解析的に分かっている例で必ず検算すること。

B.2　状態依存のベクトル場と自動微分によるリー括弧

```
import torch

def X_neom(z):
    # 中東依存度（次元 0）が高い状態ほど、中断の打撃が大きくなる例
    w = torch.sigmoid(3.0 * z[0])
    c = torch.zeros_like(z)
    c[0], c[2], c[7] = -0.7, -0.4, 0.5   # 押す方向（定数）
    return w * c                          # 押す強さは状態 z に依存

def lie_bracket(X, Y, z):
    # [X, Y]^k = X^i ∂_i Y^k − Y^i ∂_i X^k
    JX = torch.autograd.functional.jacobian(X, z, create_graph=True)  # JX[k, i] = ∂_i X^k
    JY = torch.autograd.functional.jacobian(Y, z, create_graph=True)
    return JY @ X(z) - JX @ Y(z)
```

```
def nested(X, Y):
    # 入れ子のリー括弧に渡すため、内側を「z の関数」として返す
    return lambda z: lie_bracket(X, Y, z)

# 三重リー括弧 [X_us, [X_china, X_russia]](z) の正しい組み立て方
# triple = lie_bracket(X_us, nested(X_china, X_russia), z)

def g_norm(v, g):
    # 引き戻し計量によるノルム
    return torch.sqrt(v @ g @ v)
```

B.3　測地線距離行列によるパーシステント・ホモロジー

```
import numpy as np
from ripser import ripser

names = ['saudi', 'cyber', 'indo_reg', 'uae', 'indo_bribe', 'steel', 'cbam']
n = len(names)
D = np.zeros((n, n))
for a in range(n):
    for b in range(a + 1, n):
        D[a, b] = D[b, a] = geodesic_distance(points[a], points[b])  # 測地線距離

result = ripser(D, distance_matrix=True, maxdim=2)

# 安定性の確認：点の位置に小さな摂動を加えて H1 の本数が変わらないかを見る
```

B.4　位置依存のハミルトニアンに対する一般化リープフロッグ法

運動エネルギーが $\frac{1}{2} g^{ij}(z) p_i p_j$ のように位置に依存する場合、次の陰的な 3 段の更新を用いるとシンプレクティック性が保たれる（Leimkuhler & Reich, 2004）。

$$
\begin{aligned}
p_{n+1/2} &= p_n - \tfrac{\Delta t}{2}\, \partial_z H(z_n, p_{n+1/2}) && \text{（陰的：反復で解く）} \\
z_{n+1} &= z_n + \tfrac{\Delta t}{2}\left[\partial_p H(z_n, p_{n+1/2}) + \partial_p H(z_{n+1}, p_{n+1/2}) \right] && \text{（陰的：反復で解く）} \\
p_{n+1} &= p_{n+1/2} - \tfrac{\Delta t}{2}\, \partial_z H(z_{n+1}, p_{n+1/2}) && \text{（陽的）}
\end{aligned}
$$

陰的な式は固定点反復で数回反復すれば十分に収束することが多い。長期シミュレーションでは、ハミルトニアンの相対誤差 $|H_n - H_0|/|H_0|$ を時系列で記録し、ドリフトがないことを確認する。

付録 C　コード索引

ステップ	本書の節	主なコード片
1–2	第 2 章	固有次元推定、命題 1.1 の検証（3 回）、企業状態ベクトル、異動データ、制約付き Neural ODE
3	第 3 章	引き戻し計量、条件数
4	第 4 章	スカラー曲率、測地線距離
エージェント	第 5 章	14 体のロスター、身体化・非身体化のアラート、物理的ベースライン・チェック
5	第 6 章	リー微分、リー括弧、三重リー括弧、リー括弧行列
8	第 7 章	NEOM 危機とインドネシア危機の最適制御
—	第 8 章	測地線距離行列、アラート形式
7	第 9 章	DML による因果分解
拡張	第 10 章	CBAM、パーシステント・ホモロジー、ゲーム理論、手術理論、シンプレクティック積分、層理論、座標系の変換
信頼性	第 11 章	ZKML